

Project ATHENA — Week 1 Build Session

Milestone: Build the Foundation of Your Quant Research Platform

Duration: 90–120 minutes

Objective: By the end of today's session, ATHENA will evolve from a collection of folders into a real software project with a clean architecture, reusable modules, and the first working component.

Why this matters

Most people begin by downloading stock data and experimenting in Jupyter notebooks.

Six months later they have:

Notebook_Final.ipynb

Notebook_Final_v2.ipynb

Notebook_Final_v2_NEW.ipynb

Notebook_Actually_Final.ipynb

No reusable code.

No testing.

No architecture.

That's not how engineers build systems.

Today, you're going to build the **skeleton** that everything else will plug into.

Think of it like laying the foundation before constructing a house.

Today's Deliverable

By the end, you should have:

ATHENA/

|

|— config/

|

|— data/

|

| |— raw/

|

| |— processed/

```
|   ├── features/
|   └── cache/
|
|   ├── docs/
|
|   ├── notebooks/
|
|   ├── src/
|   │   ├── ingestion/
|   │   ├── cleaning/
|   │   ├── indicators/
|   │   ├── features/
|   │   ├── strategies/
|   │   ├── portfolio/
|   │   ├── risk/
|   │   ├── backtesting/
|   │   ├── scanner/
|   │   └── utils/
|
|   ├── tests/
|
|   ├── README.md
|   ├── requirements.txt
|   └── .gitignore
```

Part 1 — Create the Project Properly (20 min)

Initialize Git.

```
git init
```

Create a virtual environment.

```
python -m venv .venv
```

Install only the essentials for now:

```
pandas
```

```
numpy
```

```
matplotlib
```

```
jupyter
```

```
pytest
```

Avoid installing dozens of packages "just in case." Keep the project lean.

Part 2 — Design the Core Architecture (25 min)

Create your first three Python modules.

market_data.py

Responsible only for obtaining data.

```
class MarketDataProvider:
```

```
    def get_daily_data(self, symbol):
```

```
        pass
```

```
    def get_intraday_data(self, symbol):
```

```
        pass
```

risk_manager.py

```
class RiskManager:
```

```
    def calculate_position_size(...):
```

```
        pass
```

```
    def calculate_risk(...):
```

```
pass
```

portfolio.py

```
class Portfolio:
```

```
    def add_position(...):
```

```
        pass
```

```
    def remove_position(...):
```

```
        pass
```

```
    def portfolio_value(...):
```

```
        pass
```

Don't implement the logic yet. Today we're designing interfaces.

Part 3 — Build Your First Working Module (30 min)

Implement your first real class:

PositionSizer

Example:

```
portfolio = 500000
```

```
risk_percent = 1
```

```
entry = 1200
```

```
stop = 1150
```

Output:

Portfolio Risk = ₹5000

Risk Per Share = ₹50

Suggested Quantity = 100

This should become your first tested utility.

Part 4 — Start Thinking Like a Software Engineer (15 min)

Create:

config/settings.py

Instead of hardcoding values.

RISK_PER_TRADE = 0.01

DEFAULT_CURRENCY = "INR"

BENCHMARK = "NIFTY50"

Future you will thank present you.

Part 5 — Create Your First Test (15 min)

Inside:

tests/

Create

test_position_size.py

Test one simple case.

Example:

Portfolio = 100000

Risk = 2%

Entry = 500

Stop = 490

Expected Quantity = 200

If you don't test early, you'll eventually spend hours debugging simple arithmetic mistakes.

Engineering Principle of the Week

Separate:

Data



Indicators



Strategies



Risk



Portfolio



Execution

Each module should know as little as possible about the others.

This makes ATHENA easier to maintain and extend.

Stretch Goal (Optional)

Create your first logging utility.

logs/

app.log

Every future module should log important events instead of printing to the console.

Example:

Loaded 5000 rows

Downloaded AAPL

Position size calculated

Good logs become invaluable when the project grows.

Deliverables Checklist

By the end of today, you should have:

- ☒ Project structure created
- ☒ Git repository initialized
- ☒ Virtual environment configured
- ☒ requirements.txt created
- ☒ README.md updated with project vision
- ☒ MarketDataProvider interface defined
- ☒ RiskManager interface defined
- ☒ Portfolio interface defined
- ☒ Working PositionSizer
- ☒ Configuration file added

-  First unit test passing
-

Retrospective Checklist

Spend five minutes answering these questions in your journal:

- Did I organize the project so future features can be added without major refactoring?
 - Did I write code that is reusable rather than notebook-specific?
 - Which module felt the most intuitive?
 - Which design decisions am I uncertain about?
 - If I revisited this project in six months, would today's structure still make sense?
-

Looking Ahead — Week 2

Next week, we'll make ATHENA useful.

Instead of placeholder interfaces, we'll connect it to real market data and build:

- A **data ingestion pipeline** for Indian and US markets.
- Data validation and cleaning.
- Local caching to avoid repeated downloads.
- The first feature engineering module.
- The foundation for a backtesting engine.

Mentor's Note

You've built ML systems before, so I'm going to push you beyond simply "writing code." From this point onward, every design decision should answer one question:

If ATHENA had one million users instead of one, would this architecture still make sense?

You don't need to build for a million users today—but thinking that way will naturally lead you toward cleaner interfaces, better abstractions, and stronger engineering habits. That's exactly the mindset that will make this platform something you're proud to use, showcase, and continue evolving long after this course ends.